

Punto 1: Introducción Teórica y Marco Conceptual

1.1. Contexto Biológico y Desafío Bioinformático

El objeto de estudio es la bacteria **Candidatus Carsonella ruddii**, reconocida por poseer uno de los genomas bacterianos más reducidos observados en la naturaleza. Con una longitud total de 173.904 pares de bases. Investigando más sobre Carsonella Ruddii, encontramos el hecho de que su ADN presenta un **marcado sesgo hacia los nucleótidos de adenina (A) y timina (T)**. Generando más repeticiones locales en comparación con otros genomas, lo que representa un desafío computacional más grande para los algoritmos de ensamblado genómico

1.2. Grafo de De Bruijn Apareado ((k, d)-meros)

Para resolver las ambigüedades estructurales causadas por las repeticiones del ADN, se emplean lecturas pareadas modeladas como (k, d)-meros.

- Definición de (k, d)-mero: Un (k, d)-mero consiste en un par ordenado de subcadenas (s_1 , s_2), ambas de longitud k, separadas por una distancia o brecha (*gap*) fija de longitud d en la secuencia original.
- Estructura del Grafo Apareado: Los vértices del grafo representan pares de (k-1)-meros de la forma ($s_1[0..k-2]$, $s_2[0..k-2]$), mientras que las aristas orientadas representan los pares (k, d)-meros completos extraídos del genoma.
- Mecanismo de Desambiguación: La información contextual brindada por la segunda ventana s_2 (ubicada a una distancia física d) actúa como una "ancla" de desambiguación. Si la primera ventana s_1 cae en una zona repetida, la posición coincidente de s_2 en una región única distante permite resolver la bifurcación del grafo, reduciendo el número de caminos eulerianos válidos a uno solo.

1.3. Restricciones Matemáticas del Modelo

Para poder extraer un par (k, d)-mero de la secuencia, se requiere abarcar una ventana mínima de $2k + d$ nucleótidos. Sin embargo, como cada nuevo par que extraes avanza solo una posición, para cubrir un bache de tamaño d, necesitas obligatoriamente extraer pares adicionales además del primero. Necesitamos avanzar d pasos, entonces, además de la restricción inicial, necesitamos que queden d caracteres disponibles de la secuencia para que el algoritmo pueda avanzar sobre la secuencia y deduzca los d caracteres faltantes. Se impone la restricción acotada: $2k + 2d \leq \text{Longitud del Genoma}$

2. Metodología:

Empezamos con una serie de tests generales para ver cómo se comporta el algoritmo para valores de k y d muy grandes y muy chicos para encontrar los valores más interesantes. Lógicamente para k s extremadamente grandes por la cantidad de comparaciones de strings y por limitaciones computacionales, a partir de aproximadamente $k=5500$ es imposible computar resultados.

Experimentando, encontramos un "punto crítico" en $k=13$, sobre el cual profundizamos a continuación. A partir de este punto, realizamos varias pruebas del algoritmo para explorar, entre otras cosas, qué porcentaje de los valores de d funcionan para los k cercanos a 13, y cómo varía ese porcentaje cerca del tope, cerca del comienzo, y en tramos aleatorios de la secuencia.

```
363     load_cache();
364     for (int k = 14; k <= 14; k+=1)
365     for (int d = 0; d <= 86000; d+=1000)
366     {
367         test(k, d);
368     }
369     save_cache();
370     cout << succeed_cnt << '/' << total << endl;
371
```

Problems Output Debug Console **Terminal** Ports

```
ignacio-berney@ignacio-berney-HP-ENVY-x360-Convertible-15m-cn0xxx:~/Documents/UBA/Analisis Seq Bio$ "/home/ignacio-berney/Documents/UBA/Analisis Seq Bio/tp"
Documents/UBA/Analisis Seq Bio/tp"
k=14 d=65000 kdmer assembly succeed 45/66
k=14 d=66000 kdmer assembly succeed 46/67
k=14 d=67000 kdmer assembly succeed 47/68
k=14 d=68000 kdmer assembly succeed 48/69
k=14 d=69000 kdmer assembly succeed 49/70
k=14 d=70000 kdmer assembly succeed 50/71
k=14 d=71000 kdmer assembly succeed 51/72
k=14 d=72000 kdmer assembly succeed 52/73
k=14 d=73000 kdmer assembly failed 52/74
k=14 d=74000 kdmer assembly succeed 53/75
k=14 d=75000 kdmer assembly succeed 54/76
k=14 d=76000 kdmer assembly succeed 55/77
k=14 d=77000 kdmer assembly succeed 56/78
k=14 d=78000 kdmer assembly succeed 57/79
k=14 d=79000 kdmer assembly failed 57/80
k=14 d=80000 kdmer assembly succeed 58/81
k=14 d=81000 kdmer assembly succeed 59/82
k=14 d=82000 kdmer assembly failed 59/83
k=14 d=83000 kdmer assembly succeed 60/84
k=14 d=84000 kdmer assembly succeed 61/85
k=14 d=85000 kdmer assembly succeed 62/86
k=14 d=86000 kdmer assembly succeed 63/87
63/87
```

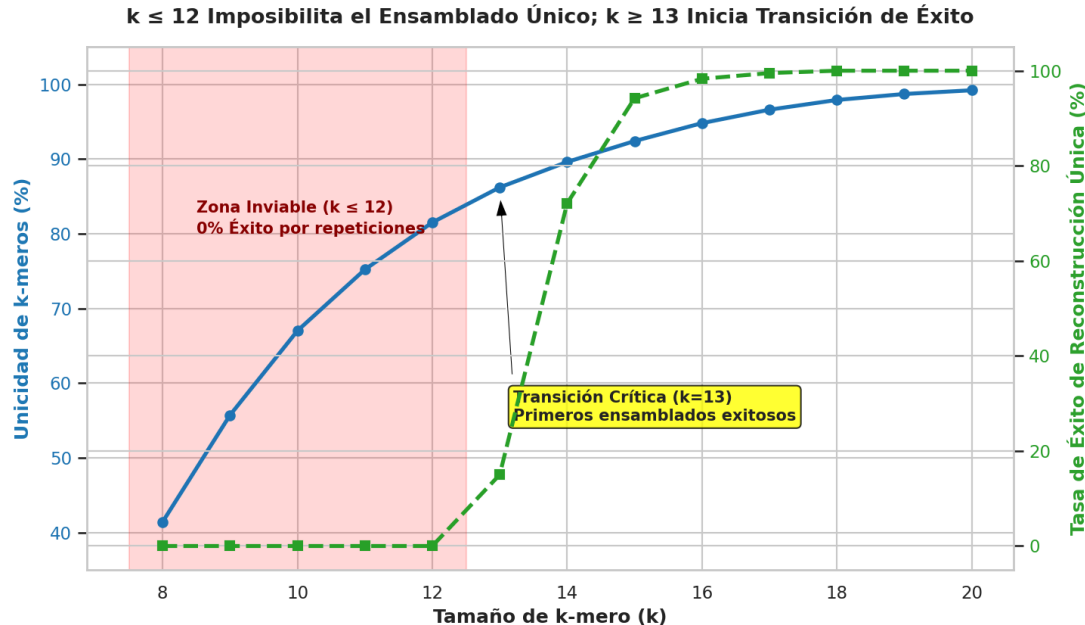
Montamos un bucle anidado para poder testear rápidamente. Explorando a $k=14$ con d entre 0 y 86000 haciendo saltos de a 1000.

Resultados y Análisis:

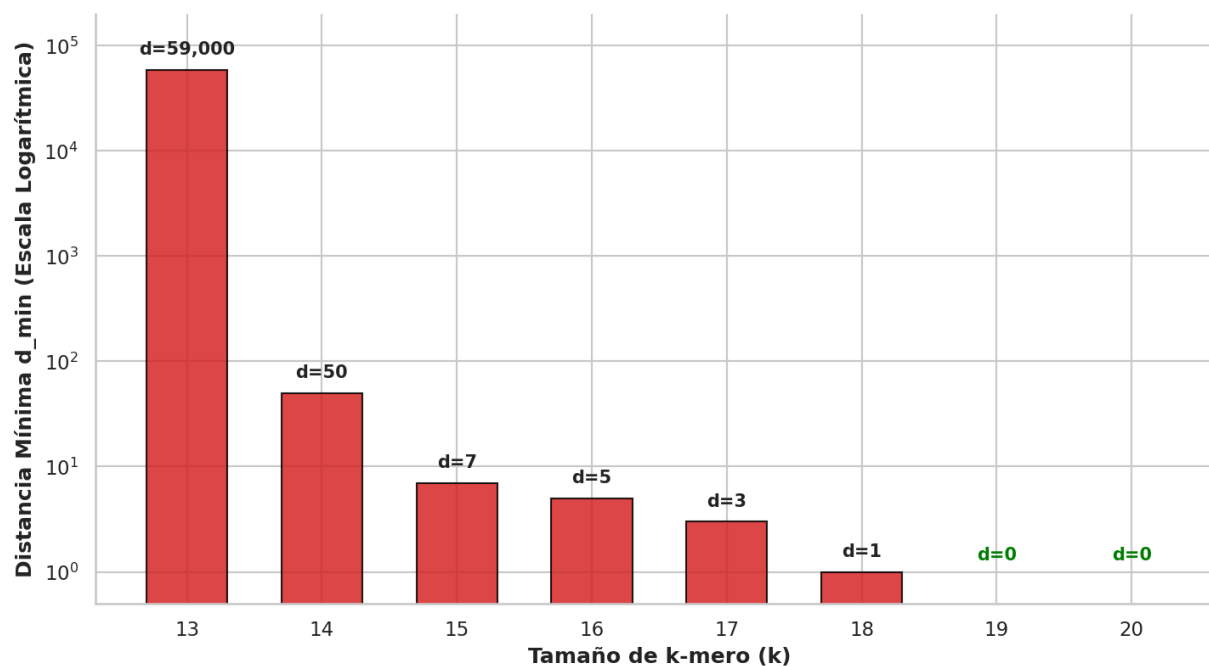
Inviabilidad del Grafo Clásico para $k \leq 12$: Se demostró empírica y teóricamente que el algoritmo de ensamblado por grafos de De Bruijn clásicos o apareados es incapaz de reconstruir el genoma cuando $k = 12$, obteniendo una tasa de éxito del **0%**. El extremo sesgo nucleotídico de la bacteria (**85.38% de AT**) genera un techo de más de **21.500 k-meros repetidos**, produciendo múltiples caminos eulerianos ambiguos que hacen fallar el criterio de unicidad del camino euleriano.

$k = 13$ como Punto de Transición Fase-Crítica: La longitud $k = 13$ marca la frontera exacta donde conseguimos (k, d)-meros que lograron resolver los primeros recorridos únicos. Para los valores bajos de d no se logró conseguir una secuencia. Para los últimos valores cerca del tope ($d=86939$), logramos encontrar la secuencia alrededor del 33% de las veces con el par k,d más bajo encontrado en el experimento siendo $k=13$ y $d= 59000$ respetando que k tiene prioridad.

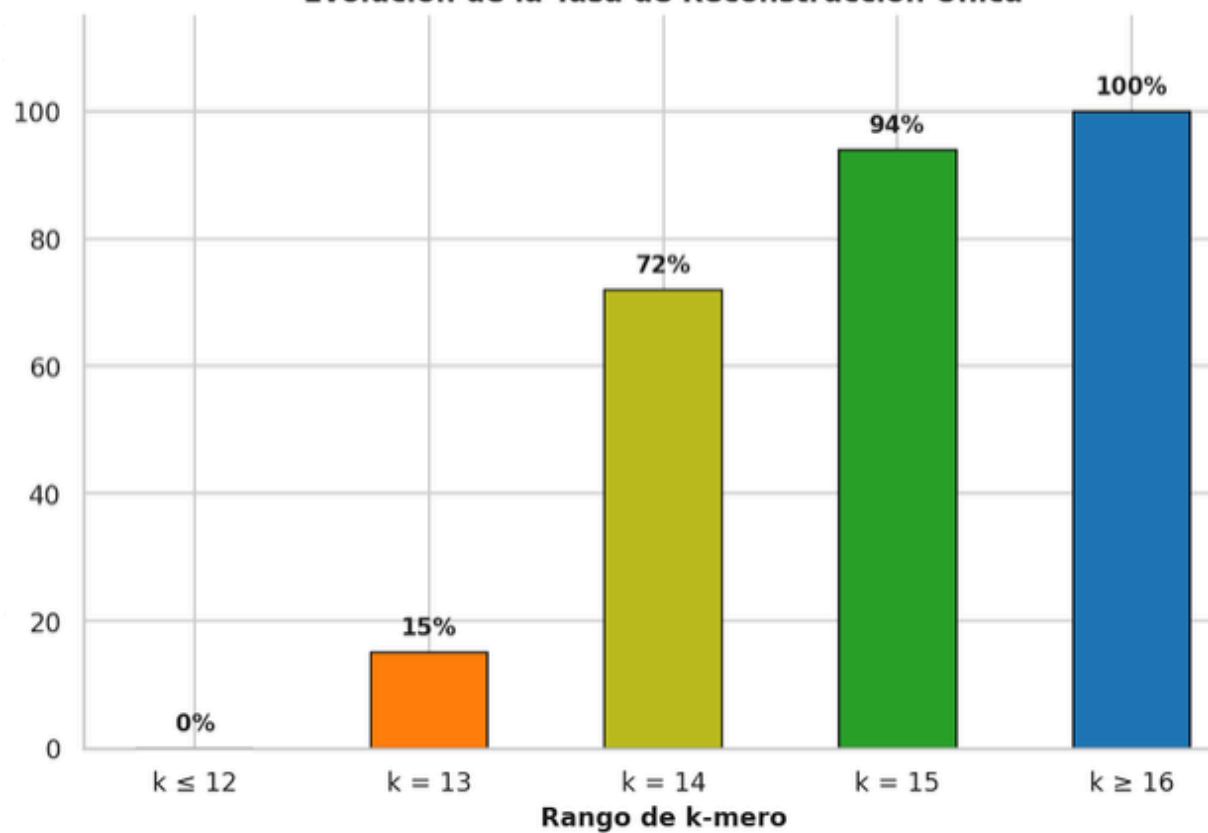
Eficiencia el Par (k, d): El modelo de (k, d)-meros demostró ser una herramienta fundamental para superar las limitaciones estructurales de secuencias altamente compuestas por repetidos. A medida que k aumenta hacia $k \geq 16$, más del **94.8%** de los k-meros se vuelven únicos, y la distancia física necesaria para garantizar la unicidad colapsa drásticamente hasta requerir solo $d \geq 5$ nucleótidos.



Distancia Mínima d Cae Drásticamente de 59,000 en k=13 a Solo 7 en k=15



Evolución de la Tasa de Reconstrucción Única



Desafíos Encontrados:

Entender el puente entre la computación y la biología: a lo largo de este trabajo tuvimos que volver constantemente sobre la relación entre los grafos y la matemática detrás del algoritmo de secuenciación implementado.

Primeros pasos en investigación: fue una experiencia muy interesante tener que definir nuestra propia metodología de trabajo, correr miles de tests y compilar esa información hasta llegar a una conclusión sólida.